

# Deploy an ASP.NET Core Application on an Ubuntu Virtual Machine

## Summary:-

In this lab, you will publish an ASP.NET Core application from Visual Studio Code, copy the published files to an Ubuntu virtual machine using SCP, run the application on the .NET runtime, configure Azure networking to allow application traffic, and verify the deployed application from a web browser.

## Let's Start:-

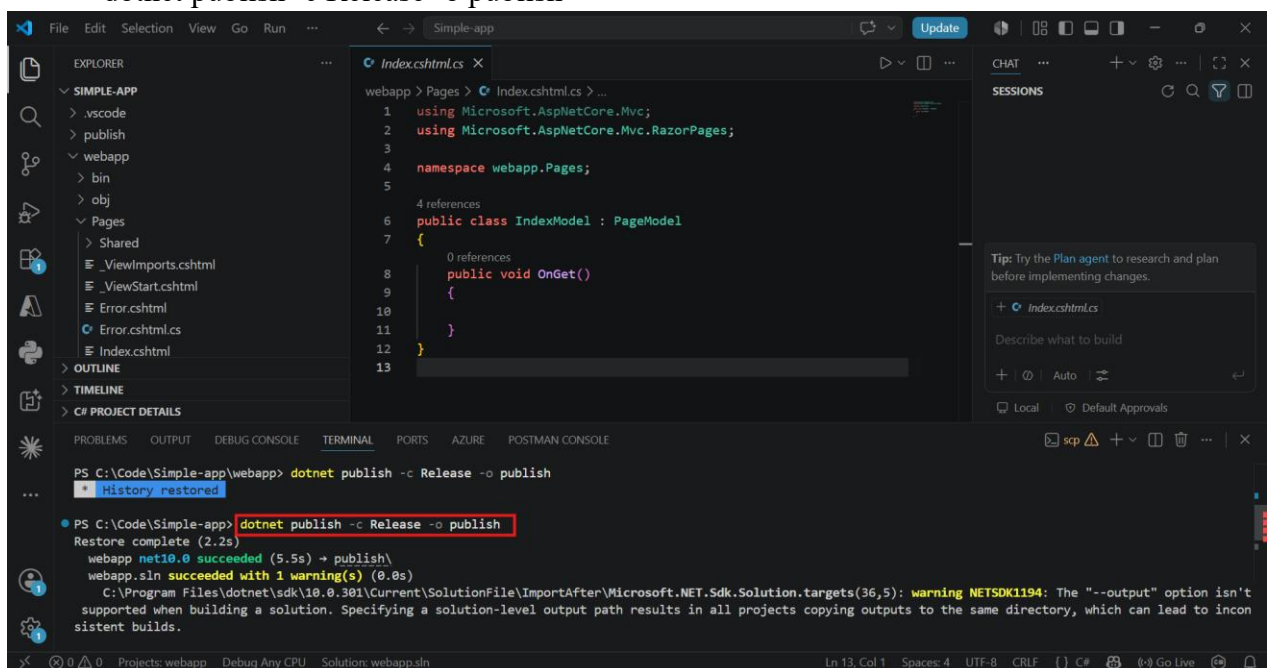
In this lab, you will:

- Publish an ASP.NET Core application.
- Copy the published application to an Ubuntu VM.
- Run the application using the .NET runtime.
- Configure a Network Security Group rule.
- Access the application through a web browser.

## Lab Steps:-

### Step 1 – Publish the ASP.NET Core Application

- Open the **project in Visual Studio Code**.
  - Open **Terminal** and publish the application.
- `dotnet publish -c Release -o publish`

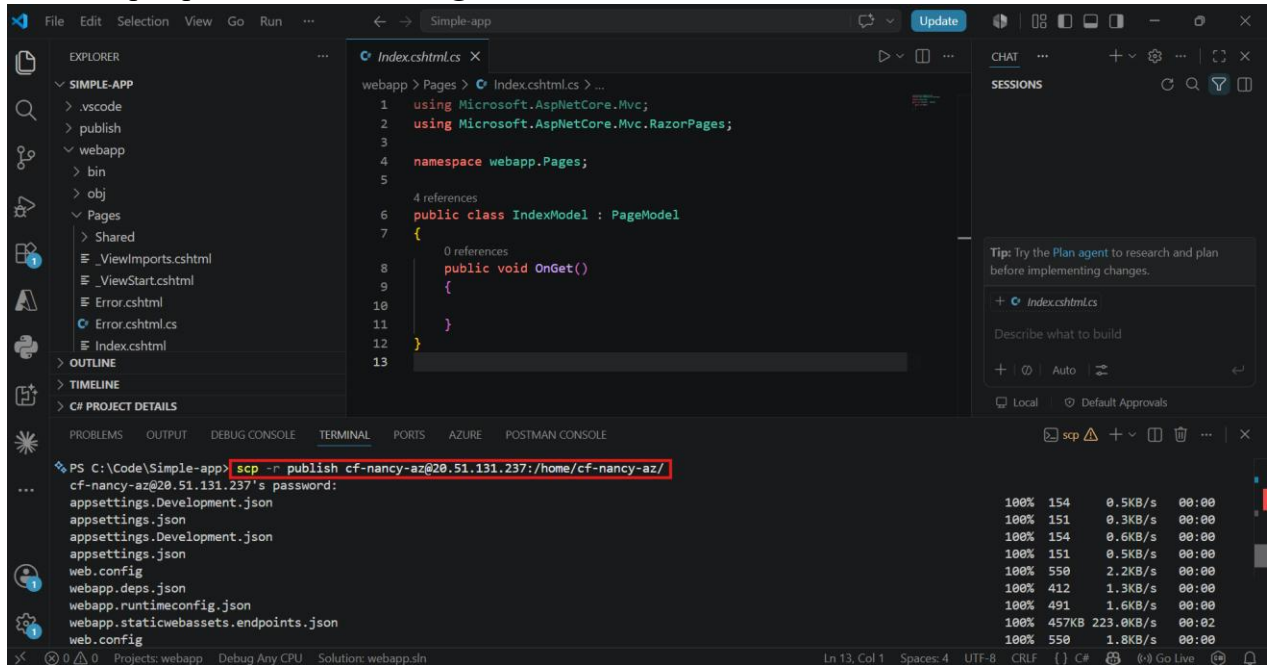


- A **publish** folder is created containing the deployment files.

## Step 2 – Copy the Application to the Ubuntu VM

- From the project directory, copy the published files to the Ubuntu VM.

`scp -r publish <username>@<Public-IP>:/home/<username>/`



- Enter the administrator password when prompted.

## Step 3 – Verify the Published Files

- Connect to the Ubuntu VM using SSH on Command Prompt.

`ssh <username>@<Public-IP>`

```
cf-nancy-az@web-dev-eus-01 X + v
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\HP> ssh cf-nancy-az@20.51.131.237
cf-nancy-az@20.51.131.237's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1020-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Jul 15 10:30:59 UTC 2026

System load:  0.01          Temperature:    71.8 C
Usage of /:   7.3% of 28.02GB Processes:      128
Memory usage: 4%           Users logged in: 1
Swap usage:   0%           IPv4 address for eth0: 10.1.0.4

Expanded Security Maintenance for Applications is not enabled.

11 updates can be applied immediately.
11 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Jul 15 10:25:24 2026 from 49.36.185.34
```

- Verify the copied folder.  
ls
- Navigate into the publish directory.  
cd publish  
dir

```
Last login: Wed Jul 15 10:25:24 2026 from 49.36.185.34
cf-nancy-az@web-dev-eus-01:~$ ls
publish
cf-nancy-az@web-dev-eus-01:~$ cd publish
cf-nancy-az@web-dev-eus-01:~/publish$ dir
appsettings.Development.json  publish          webapp.dll  webapp.runtimeconfig.json
appsettings.json              web.config      webapp.exe  webapp.staticwebassets.endpoints.json
dotnet-install.sh             webapp.deps.json webapp.pdb  wwwroot
cf-nancy-az@web-dev-eus-01:~/publish$ dotnet webapp.dll --urls http://0.0.0.0:5000
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://0.0.0.0:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /home/cf-nancy-az/publish
```

```
cf-nancy-az@web-dev-eus-01 X + v
Selecting previously unselected package libltng-ust1t64:amd64.
Preparing to unpack .../4-libltng-ust1t64_2.13.7-1.1ubuntu2_amd64.deb ...
Unpacking libltng-ust1t64:amd64 (2.13.7-1.1ubuntu2) ...
Selecting previously unselected package dotnet-runtime-10.0.
Preparing to unpack .../5-dotnet-runtime-10.0_10.0.9-0ubuntu1~24.04.1_amd64.deb ...
Unpacking dotnet-runtime-10.0 (10.0.9-0ubuntu1~24.04.1) ...
Selecting previously unselected package aspnetcore-runtime-10.0.
Preparing to unpack .../6-aspnetcore-runtime-10.0_10.0.9-0ubuntu1~24.04.1_amd64.deb ...
Unpacking aspnetcore-runtime-10.0 (10.0.9-0ubuntu1~24.04.1) ...
Setting up libltng-ust-common1t64:amd64 (2.13.7-1.1ubuntu2) ...
Setting up libltng-ust-ctl5t64:amd64 (2.13.7-1.1ubuntu2) ...
Setting up dotnet-host-10.0 (10.0.9-0ubuntu1~24.04.1) ...
Setting up dotnet-hostfxr-10.0 (10.0.9-0ubuntu1~24.04.1) ...
Setting up libltng-ust1t64:amd64 (2.13.7-1.1ubuntu2) ...
Setting up dotnet-runtime-10.0 (10.0.9-0ubuntu1~24.04.1) ...
Setting up aspnetcore-runtime-10.0 (10.0.9-0ubuntu1~24.04.1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
cf-nancy-az@web-dev-eus-01:~$ dir
publish
cf-nancy-az@web-dev-eus-01:~$ cd publish
cf-nancy-az@web-dev-eus-01:~/publish$ dir
appsettings.Development.json  publish  webapp.deps.json  webapp.exe  webapp.runtimeconfig.json  wwwroot
appsettings.json              web.config  webapp.dll        webapp.pdb  webapp.staticwebassets.endpoints.json
```

## Step 4 – Run the ASP.NET Core Application

- Start the application.  
dotnet WebApp.dll --urls <http://0.0.0.0:5000>

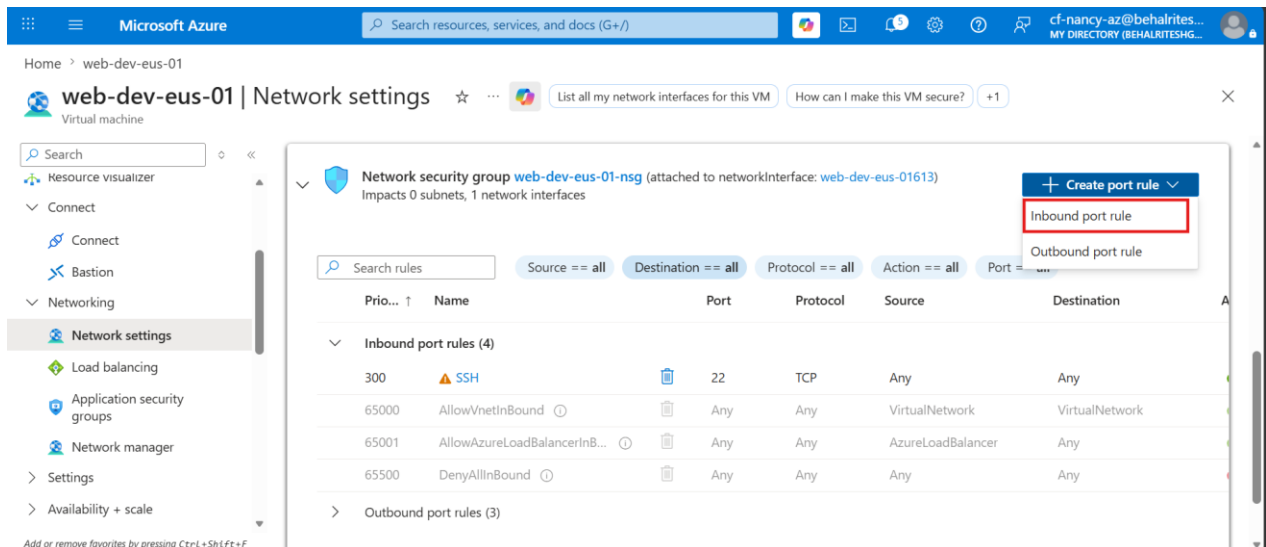
```
cf-nancy-az@web-dev-eus-01:~/publish$ dir
appsettings.Development.json  publish  webapp.dll  webapp.runtimeconfig.json
appsettings.json              web.config  webapp.exe  webapp.staticwebassets.endpoints.json
dotnet-install.sh             webapp.deps.json  webapp.pdb  wwwroot
cf-nancy-az@web-dev-eus-01:~/publish$ dotnet webapp.dll --urls http://0.0.0.0:5000
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://0.0.0.0:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /home/cf-nancy-az/publish
```

- The application begins listening on **Port 5000**.

## Step 5 – Allow Port 5000 in Azure

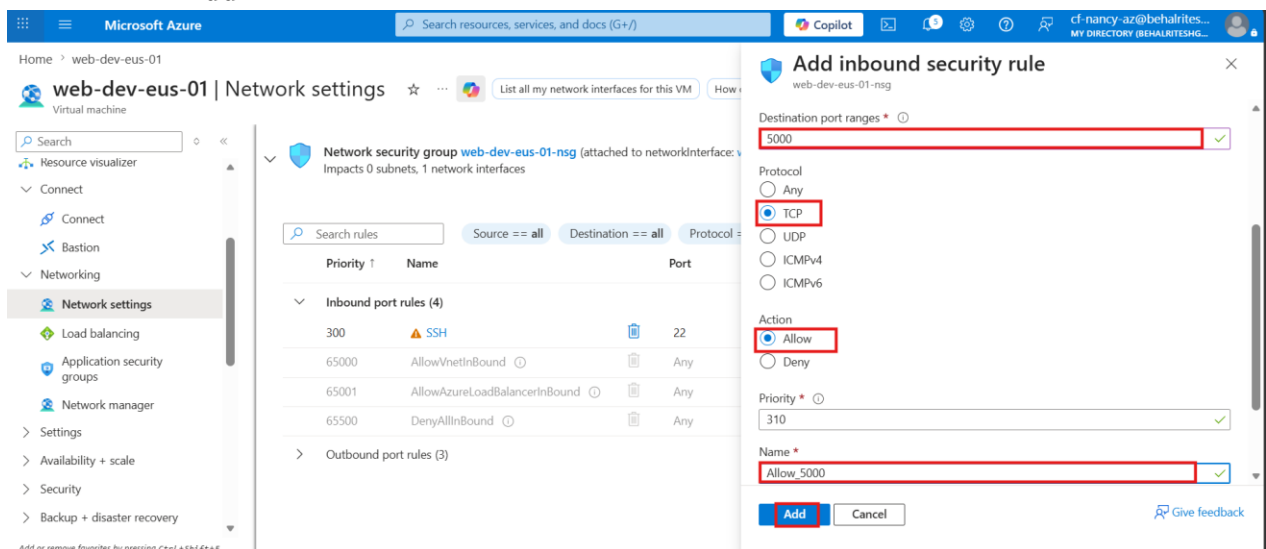
In the Azure Portal:

- Open the Virtual Machine.
- Select **Networking**.
- Click **Add inbound port rule**.



- Configure:
 

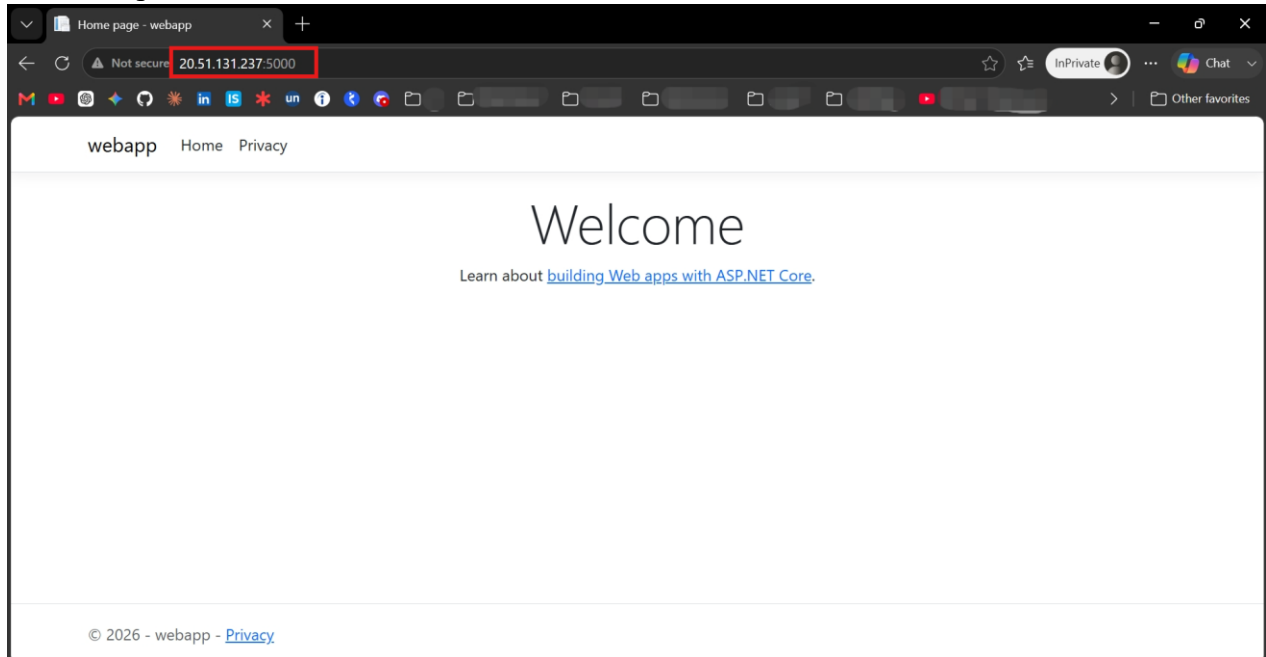
| Setting          | Value      |
|------------------|------------|
| Source           | Any        |
| Destination Port | 5000       |
| Protocol         | TCP        |
| Action           | Allow      |
| Name             | Allow-5000 |
- Click Add.



## Step 6 – Access the Application

- Open a web browser.
- Browse to:

http://<Public-IP>:5000



- The ASP.NET Core welcome page should be displayed.

## Verification

Verify that:

- The publish folder is copied successfully.
- The ASP.NET Core application starts without errors.
- Azure Network Security Group allows inbound traffic on Port 5000.
- The application is accessible through the browser using the VM's public IP address.